Designing an AI video pipeline: the LLM chooses, the code computes

2026-10-13

I want a repeatable way to turn a topic and a few sources into a three-minute narrated explainer video, with an AI model doing the judgement work and ordinary code doing the rest. This post describes the pipeline I settled on. The design does not depend on any one vendor: every AI service sits behind a small interface and can be swapped. I use Claude Code as the worked example for orchestration, but the same structure works with any agent framework, or with plain scripts and a person at the keyboard.

1 The principle: the LLM chooses, the code computes

A language model is good at deciding what a script should say, where a scene should cut, and whether a picture looks right. It is unreliable at arithmetic and at obeying hard constraints such as "no gaps between timestamps". So the pipeline splits the work:

- **The model decides:** the script, the cut points, the visual prompts, the quality review.
- **Code computes:** speech synthesis, timing, cost estimates, validation and rendering.

Everything below follows from that split.

2 Scope

I fixed the scope first, because it drives later choices.

- **Format: 16:9, about three minutes.** A full run is roughly 60 scenes, so a failed run is expensive. I develop against a 30–60 second cut and

only then run the full length. Vertical video is a second project, not a setting: captions, framing and prompts all change.

- **Language: English.** The speech, alignment and image tools I want all work well in English.
- **Word budget: about 155 words per minute.** Three minutes is roughly 465 words, and the validator holds the script to within a few percent of that.

3 The pipeline

```
brief → script → fact-check → [approve script]
      → speech → align → beatsheet → validate
      → cost estimate → [approve spend]
      → assets → visual QA → render → final QA and packaging
```

Each stage reads files and writes files. A manifest records, per stage, the input hash and the output, so a re-run skips anything unchanged.

3.1 1. Script

The model receives the brief (topic, audience, sources, target length) and a style file. It returns structured sections, each with its spoken text and the factual claims it makes. Code, not the model, joins the sections into the full voiceover, so the two can never drift apart. A validator checks the word budget, a hook of about eight spoken words, and that the spoken text has no brackets or stage directions.

A separate fact-checking pass compares each claim with the sources and flags anything unsupported before any money is spent.

3.2 2. Speech

The script is synthesised **section by section** and then joined. That gives section boundaries for free and lets me redo one bad section alone. Numbers, units and acronyms are normalised first, with a small pronunciation lexicon for names the engine gets wrong, and the final track is loudness-normalised.

The speech engine sits behind one function: text and voice in, audio and (if available) word timings out. I develop on a free engine, edge-tts, because it costs nothing to iterate and returns word timings I can cross-check. It uses an unofficial endpoint with unclear commercial terms, so I treat it as a devel-

opment tool and plan to switch to a commercial engine such as ElevenLabs or Fish Audio before publishing anything.

3.3 3. Alignment: audio is the master clock

I already know exactly what the voice says, so I use **forced alignment** against the known script (for example WhisperX or stable-ts) rather than transcribing the audio blind. The output is `words.json`: every word with a start and end time. Every visual later is anchored to those times. Subtitle files are just a derived export, because an SRT keeps only segment-level times and would throw the word-level clock away. Captions are chunked separately for reading (at most two lines of about 42 characters), independently of how scenes are cut.

3.4 4. Beatsheet: the model returns cut points, not timestamps

The director step sees the script and a transcript numbered by word (`0: Why`, `1: do`, ...), and returns scenes that refer to word indices:

```
{"scene": 1, "first_word": 0, "last_word": 8,
 "visual_type": "still", "visual_prompt": "Close-up of a confused cartoon charac
 "motion_prompt": "slow push-in", "on_screen_text": null}
```

A short validator then does the arithmetic. It checks that scenes cover every word with no gaps, converts indices to times, snaps to the frame grid, enforces two to five seconds per scene, and confirms the total equals the audio length. If validation fails, the error text goes back to the model once. A second failure stops the run and asks me.

Two details keep the prompts clean. Style words ("flat vector illustration") live only in the style file and are appended at generation time, so they cannot conflict with scene prompts. And a prompt describes subject and action only: literal and physical, not abstract.

3.5 5. Cost, and the second gate

A script counts the images, video seconds and speech characters, multiplies by a price file, and writes an estimate. If it is over the cap, the run fails. Only after I approve it does the marker file appear that lets the paid steps run.

3.6 6. Assets

A three-minute video at three seconds a scene is about 60 scenes. Generating video for every one is slow and costly, so most scenes are a still image with a camera move, and a small, capped number are real generated video clips. Stock footage, where used, comes from a licensed source with the licence recorded.

For images I use a hosted Flux model through an API, behind one `generate_image()` function. Hosting matters because generating 60 images on a laptop is slow, and the function boundary means I can change model or provider without touching anything else. Each scene gets a fixed seed, recorded in the manifest, plus a shared character and style sheet to keep the look consistent.

One rule matters more than the model choice: **image models garble text, so text is never generated.** Scenes that are really diagrams, titles or interface mock-ups make no image call at all. They are drawn by the compositor from structured data.

3.7 7. Visual QA

The pipeline builds a contact sheet, one labelled frame per scene, and a model that can read images checks it for style consistency, legibility, artefacts and whether each picture matches its prompt. Failed scenes are regenerated individually, at most twice, and then come to me.

3.8 8. Render

For assembly I use Remotion, a React-based video renderer, for compositing, and ffmpeg for audio mixing, loudness and checks. The reason is the visual types. Animated word-highlight captions, transitions and text or interface scenes are motion graphics. In ffmpeg they turn into fragile filter graphs; in Remotion they are components that read the compiled JSON. The renderer reads only `beats.json`, `words.json` and the asset folder, with no network and no model, so a render is reproducible, and anything that honours that contract could replace it. Remotion's licence is free for individuals and small teams and paid for larger companies, so I will read the terms against my situation before publishing.

The final stage checks duration, loudness, black frames and caption safe areas, and the model drafts the packaging: title options, description, chapters from the section boundaries, a thumbnail concept and the AI-content disclosure line.

4 Two gates, and the hook that enforces them

1. **Approve the script**, after the fact-check and before any speech is synthesised.
2. **Approve the spend**, after the cost estimate and before any paid generation.

Editing the script after the first gate invalidates every later stage and removes both approvals. Two small guards enforce this. One blocks the paid steps unless the approval marker exists and the estimate is under the cap. The other validates each JSON file against its schema when it is written, so a malformed file cannot reach a paid stage. A guard should earn its place by naming the specific irreversible harm it prevents; here that harm is spending money without approval or on bad input.

5 Mapping it onto an agent such as Claude Code

Nothing above requires Claude Code, but it fits well:

- **Skills** hold the two judgement prompts, the script-writer and the beat-director, each pointing at its JSON schema.
- **Subagents** run the fact-checker and the visual-QA reviewer. Keeping contact sheets and source dumps in separate contexts stops them filling the main session.
- **Hooks** implement the two guards. They run whatever the model decides, which is why they, not instructions in a prompt, protect the budget.
- **Plan mode** is the natural place for the two approvals.
- **Slash commands** scaffold a job, resume it from the manifest, and print its status and spend.
- **Headless mode** can run the two model stages unattended in a batch.

In another stack the same roles are filled by whatever your framework calls prompts, sub-agents, middleware and human-in-the-loop checkpoints. I keep the pipeline in its own repository, separate from this site, since this site deliberately has no build machinery.

6 What to check before publishing a video

- **Voices.** Use stock voices only. Do not clone a real person without documented consent.
- **Licences.** Record the licence of every music, effect and stock-footage asset, and confirm that the image model's terms allow commercial use of its output.
- **Disclosure.** Add the platform's AI-content label and put the line in the description.
- **Drift.** If a speech or image model version changes, outputs may not reproduce. The manifest records versions so I notice.

7 Build order and definition of done

Build the parts that cost nothing first: schemas, the beatsheet validator and test fixtures. Then speech and alignment on a 30-second sample, then the model stages as a dry run that ends at the cost estimate. Only then add image generation and the Remotion scenes, one component at a time, starting with stills and captions because they cover most scenes.

The pipeline is done when one full video comes out of a brief with exactly two human approvals, no hand-edited timestamps, a resumable manifest, and actual spend within 10% of the estimate. When I have built it I will write a follow-up on what broke.